

Coin Dash Pt. 3

Prepared by: Rene Andre Bedonia Jocsing

Published: February 06, 2026

Session Objectives

- Build HUD using Control nodes and containers
- Implement signal-driven UI updates
- Create title screen with menu state management
- Add sound effects for game events
- Complete full game loop with restart capability

Part 1: HUD Scene

The HUD displays score, timer, and messages during gameplay.

Scene Structure

Node Hierarchy:

```
1 HUD (CanvasLayer)
2 |-- Timer
3 |-- MarginContainer
4 |   |-- Score (Label)
5 |   |-- Time (Label)
6 |-- Message (Label)
7 |-- StartButton (Button)
```

Build Steps

1. Create Root and Timer

- New scene, root: CanvasLayer named HUD
- Add Timer child
- Timer: Wait Time = 2, One Shot = On

Design Decision: CanvasLayer

CanvasLayer renders UI above game objects regardless of camera position. Essential for HUDs that stay fixed on screen.

2. Score and Time Display

- Add MarginContainer
- Layout: Top Wide
- Theme Overrides/Constants: Right = 10, Left = 10, Top = 10, Bottom = 10
- Add two Label children: Score and Time

3. Configure Score/Time Labels

- Score: Text = “0”, Horizontal Alignment = Left
- Time: Text = “0”, Horizontal Alignment = Right
- Both: Vertical Alignment = Center
- Both: Label Settings, New LabelSettings
- Font: res://assets/Kenney Bold.ttf
- Size: 48
- Outline: Size = 16, Color = black

Common Pitfalls

Font not appearing?

- Verify font file in assets folder
- Check LabelSettings is actually assigned (not just created)
- Confirm Size is set (default 16 is too small)

4. Message Label

- Add Label to HUD root named Message
- Layout: Center Wide
- Text: “Coin Dash!”
- Horizontal Alignment: Center
- Label Settings: Same font, Size = 72

5. Start Button

- Add Button to HUD root named StartButton
- Layout: Center Bottom
- Text: “Start”
- Label Settings: Same font, Size = 48

HUD Script

```
1 extends CanvasLayer
2
3 signal start_game
4
5
6 func update_score(value: int) -> void:
7     $MarginContainer/Score.text = str(value)
8
9
10 func update_timer(value: int) -> void:
11     $MarginContainer/Time.text = str(value)
12
13
14 func show_message(text: String) -> void:
15     $Message.text = text
```

```
16     $Message.show()
17     $Timer.start()
18
19
20 func _on_timer_timeout() -> void:
21     $Message.hide()
22
23
24 func _on_start_button_pressed() -> void:
25     $StartButton.hide()
26     $Message.hide()
27     start_game.emit()
28
29
30 func show_game_over() -> void:
31     show_message("Game Over")
32     await $Timer.timeout
33     $StartButton.show()
34     $Message.text = "Coin Dash!"
35     $Message.show()
```

Design Pattern: Async UI Flow

`await $Timer.timeout` pauses execution until the timer signal fires. This creates a clean sequence:

1. Show “Game Over” (2 seconds via timer)
2. Timer fires timeout
3. Show title and start button

Without `await`, all steps execute instantly.

Connect Signals:

- Timer timeout to `_on_timer_timeout`
- StartButton pressed to `_on_start_button_pressed`

Part 2: Integrate HUD with Main

Add HUD to Main

- Open `main.tscn`
- Add instance of `hud.tscn`

Update Main Script

Connect HUD `start_game` signal:

- Select HUD instance
- Node tab, find `start_game` signal
- Connect to `new_game` method in Main

Update new_game:

```
1 func new_game() -> void:
2     playing = true
3     level = 1
4     score = 0
5     time_left = playtime
6
7     $HUD.update_score(score)
8     $HUD.update_timer(time_left)
9     $HUD.show_message("Get Ready!")
10
11     $Player.start()
12     $Player.show()
13
14     await $HUD/Timer.timeout
15     $GameTimer.start()
16     spawn_coins()
```

****Update _on_game_timer_timeout:****

```
1 func _on_game_timer_timeout() -> void:
2     time_left -= 1
3     $HUD.update_timer(time_left)
4     if time_left <= 0:
5         game_over()
```

Update game_over:

```
1 func game_over() -> void:
2     playing = false
3     $GameTimer.stop()
4     get_tree().call_group("coins", "queue_free")
5     $HUD.show_game_over()
6     $Player.die()
```

****Update _on_player_pickup:****

```
1 func _on_player_pickup(type: String) -> void:
2     match type:
3         "coin":
4             score += 1
5             $HUD.update_score(score)
6         "powerup":
7             time_left += 5
8             $HUD.update_timer(time_left)
```

****Update _process for level advancement:****

```
1 func _process(delta: float) -> void:
2     if not playing:
3         return
4
5     if get_tree().get_nodes_in_group("coins").size() == 0:
6         level += 1
7         time_left += 5
8         $HUD.update_timer(time_left)
9         $HUD.show_message("Level %s" % level)
10        spawn_coins()
```

Part 3: Sound Effects

Add AudioStreamPlayer nodes to play sounds for game events.

Add to Main Scene

- Add 3 AudioStreamPlayer nodes to Main
- Names: CoinSound, LevelSound, EndSound
- Drag audio files from res://assets/audio/:
- CoinSound: Coin.wav
- LevelSound: Level.wav
- EndSound: EndSound.wav
- PowerupSound: PowerupSound.wav
- HitSound: Hit.wav

Audio Best Practices

- Use .wav for short effects (better loop points)
- Use .ogg for longer sounds/music (smaller file size)
- Adjust Volume dB if too loud (try -10 to start)

Trigger Sounds in Code

****In _on_player_pickup:****

```
1 func _on_player_pickup(type: String) -> void:
2     match type:
3         "coin":
4             $CoinSound.play()
5             score += 1
6             $HUD.update_score(score)
7         "powerup":
8             time_left += 5
9             $HUD.update_timer(time_left)
10            $PowerupSound.play()
```

****In _process (level up):****

```
1 if get_tree().get_nodes_in_group("coins").size() == 0:
2     level += 1
3     time_left += 5
4     $LevelSound.play()
5     $HUD.update_timer(time_left)
6     $HUD.show_message("Level %s" % level)
7     spawn_coins()
```

In game_over:

```
1 func game_over() -> void:
2     playing = false
3     $GameTimer.stop()
4     $EndSound.play()
5     get_tree().call_group("coins", "queue_free")
6     $HUD.show_game_over()
7     $Player.die()
```

Part 4: Visual Polish*Coin Animation Shimmer*

Add Timer to Coin scene for randomized animation start.

Update coin.gd:

```
1 func _ready() -> void:
2     $Lifetime.wait_time = randf_range(3, 8)
3     $Lifetime.start()
4
5
6 func _on_lifetime_timeout() -> void:
7     $AnimatedSprite2D.frame = 0
8     $AnimatedSprite2D.play()
```

Configure AnimatedSprite2D:

- Animation Looping: Off
- Speed: 12 FPS

Connect Lifetime signal:

- Connect timeout to _on_lifetime_timeout

Obstacles

Create obstacles that end the game when touched.

Scene Setup:

```
1 Obstacle (Area2D)
2 |-- Sprite2D
3 |-- CollisionShape2D
```

Configuration:

- Sprite2D: Texture = res://assets/cactus.png
- CollisionShape2D: RectangleShape2D covering sprite
- Layer: obstacles (layer 3)
- Mask: player
- Node Tab, Groups: Add obstacles

No script needed — player already checks for obstacles group.

Add to Main:

- Add instance to Main scene
- Position manually in viewport

Coin/Powerup Overlap Prevention

Prevent spawning on obstacles.

Update coin.gd and powerup.gd:

```
1 func _on_area_entered(area: Area2D) -> void:
2     if area.is_in_group("obstacles"):
3         position = Vector2(
4             randi_range(0, screensize.x),
5             randi_range(0, screensize.y)
6         )
```

Connect signals:

- Coin/Powerup area_entered to _on_area_entered

Testing Checklist

HUD

- Score updates when collecting coins
- Timer counts down every second
- “Get Ready” message displays at game start
- “Game Over” message displays, then shows menu
- Start button restarts game

Audio

- Coin sound plays on collection
- Level up sound plays when advancing
- Game over sound plays on death/timeout

Polish

- Coins shimmer at random intervals
- Obstacles prevent coin spawning on top
- Hitting obstacle ends game

Common Pitfalls

Common issues:

- HUD not updating: Check signals connected correctly
- Sounds not playing: Verify audio files imported
- Start button doesn't work: Check signal emits to `new_game`
- Coins still spawn on obstacles: Verify `area_entered` connected

Complete Main Script

```
1  extends Node
2
3  @export var coin_scene: PackedScene
4  @export var powerup_scene: PackedScene
5  @export var playtime: int = 30
6
7  var level: int = 1
8  var score: int = 0
9  var time_left: int = 0
10 var screensize: Vector2 = Vector2.ZERO
11 var playing: bool = false
12
13
14 func _ready() -> void:
15     screensize = get_viewport().get_visible_rect().size
16     $Player.screensize = screensize
17     $Player.hide()
18
19
20 func new_game() -> void:
21     playing = true
22     level = 1
23     score = 0
24     time_left = playtime
25
26     $HUD.update_score(score)
27     $HUD.update_timer(time_left)
28     $HUD.show_message("Get Ready!")
```



```
29
30     $Player.start()
31     $Player.show()
32
33     await $HUD/Timer.timeout
34     $GameTimer.start()
35     spawn_coins()
36
37
38 func spawn_coins() -> void:
39     for i in level + 4:
40         var coin := coin_scene.instantiate()
41         add_child(coin)
42         coin.screen_size = screen_size
43         coin.position = Vector2(
44             randi_range(0, screen_size.x),
45             randi_range(0, screen_size.y)
46         )
47
48
49 func _process(delta: float) -> void:
50     if not playing:
51         return
52
53     if get_tree().get_nodes_in_group("coins").size() == 0:
54         level += 1
55         time_left += 5
56         $LevelSound.play()
57         $HUD.update_timer(time_left)
58         $HUD.show_message("Level %s" % level)
59         spawn_coins()
60
61
62 func _on_game_timer_timeout() -> void:
63     time_left -= 1
64     $HUD.update_timer(time_left)
65     if time_left <= 0:
66         game_over()
67
68
69 func game_over() -> void:
70     playing = false
71     $GameTimer.stop()
72     $EndSound.play()
73     get_tree().call_group("coins", "queue_free")
74     $HUD.show_game_over()
75     $Player.die()
76
77
78 func _on_player_pickup(type: String) -> void:
79     match type:
```

```
80     "coin":
81         $CoinSound.play()
82         score += 1
83         $HUD.update_score(score)
84     "powerup":
85         time_left += 5
86         $HUD.update_timer(time_left)
87         $PowerupSound.play()
```

Architecture Review

What We Built

- Container-based UI layout system
- Signal-driven state management (menu/game/gameover)
- Async UI flows using await
- Audio integration for game events
- Complete game loop with restart

Key Takeaways

1. **Containers:** Automatic layout beats manual positioning
2. **CanvasLayer:** Decouples UI from game world
3. **Await:** Clean async sequencing without explicit timing
4. **Groups:** Flexible object categorization and batch operations

Extension Ideas

- High score persistence (file I/O)
- Multiple obstacle types
- Particle effects on coin collection
- Camera shake on game over